Information Leakage from Prices in Query-based Data Markets

Teng Tu*[1] Huanhuan Peng*[2] Xiaoye Miao*[3] Guanjie Cheng‡[4] Shuiguang Deng†[5] Jianwei Yin†[6]
*Center for Data Science, Zhejiang University, Hangzhou, China
‡School of Software Technology, Zhejiang University, Ningbo, China
†College of Computer Science, Zhejiang University, Hangzhou, China
Email: {tuteng[1], hhpeng[2], miaoxy[3], chengguanjie[4], dengsg[5]}@zju.edu.cn zjuyjw[6]@cs.zju.edu.cn

***Abstract*—Query-based pricing enables personalized data acquisition for buyers, exhibiting strong potential in data markets. Prior studies design arbitrage-free price functions to accelerate query price computation. However, the proposed prices often correlate with query answers, leaking information about the underlying data. In this paper, we study information leakage and attack surfaces in query pricing with the goal of designing mitigation strategies that mislead attackers at the pricing stage, which maintains bounded overpayment for buyers. We introduce three price-only inference attacks that derive sensitive information at no financial cost, when investigating how query prices leak information about data existence and cardinality. To reduce the leakage risk, we propose PerPricer, a perturbation-based pricing framework that releases masked prices computed on perturbed data with bounded overpayment guarantees for buyers. It is suitable for existing query pricing mechanisms. Extensive experiments on real-world datasets and benchmarks verify the existence of information leakage in query pricing and the effectiveness of the proposed PerPricer.**

***Index Terms*—Data pricing, Data marketplace**

I. Introduction

Data serves as the foundational resource for data-driven applications, generating substantial economic value and innovation [23], [28]. The global big data market, valued at $327.26 billion in 2023, is projected to exhibit a 14.9% compound annual growth rate until 2030 [53]. This growth has spurred the emergence of specialized data markets (e.g., AWS data marketplace [1]), enabling efficient data transactions across diverse individuals and parties.

Query-based pricing [5], [29] lets buyers issue ad hoc queries and pay only for the tuples they need, making it attractive for data markets. As illustrated in Fig. 1, each customer purchases a tailored slice of the database—for instance, issuing `select * from Movie where year between 1970 and 2010` to focus on a specific era. This per-query charge avoids paying for irrelevant data, delivering flexibility and lower cost to customers.

Prior query pricing studies can be summarized into usage-based pricing (UBP) and information-based pricing (IBP). UBP prices a query by the minimum-cost combination of tuples or views that can materialize its answer, yielding arbitrage-free prices for SPJ queries [29]–[31], [37], [51]. IBP instead treats the database as a random variable and prices the information a query reveals about the true instance, covering both SPJ and aggregates through information-theoretic guarantees [16]–[18], [34], [35]. Both of them are practical for real-world query trading, providing theoretical guarantees and efficient pricing algorithms.

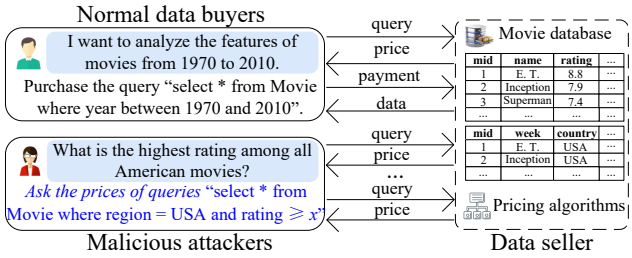

Fig. 1. Examples of query pricing and information leakage

However, the tight coupling between price and answer lets attackers extract data without payment. Fig. 1 shows that by probing prices of parametric queries (e.g., `select * from Movie where region='USA' and rating` $\geq x$) and finding the largest x with a non-zero quote, one recovers the maximum rating of American movies. This free-riding threat makes leakage-resilient pricing mechanisms imperative for secure data commercialization.

Addressing information leakage in query-based data markets faces two challenges. The first one is how to extract the critical pricing patterns leaking data information, which requires a comprehensive pricing behavior analysis of existing literature. Second, the query price plays an important role in data trading, which cannot be arbitrarily or randomly assigned. Hence, mitigating information leakage must consider the impact on participants in data markets. Moreover, the mitigation method should cause less overpayment for data buyers to maintain their interests, while all leakage mitigation approaches have overpayment. In addition, none of query pricing studies formally tackle the above problems [16], [17], [29]–[31], [34], [37], [51], while information leakage studies across other fields (e.g., query processing [6], [13] and machine learning models [9], [48]) are infeasible due to the different domains.

In this paper, we make the first systematic investigation of pricing-induced information leakage in data markets. We formalize two fundamental inference types, i.e., data existence (aka. whether a value v exists in the database) and data cardinality (aka. occurrence frequency of v). They serve as

Teng Tu and Huanhuan Peng contributed equally.

the fundamental basis to infer complex information (e.g., extreme values) and have been extensively examined in prior studies [6], [9], [13], [26]. Then, we propose four effective algorithms *Naive*, MI, Cardinality Inference via `LIMIT` clause (CIL), and Cardinality Inference via `HAVING` clause (CIH) to infer information from various pricing patterns of provenance-based and information-based pricing. *Naive* derives the existence of single-relation values from the zero prices of queries. MI leverages the *arbitrage-free* property of query pricing (i.e., for any query set $\{Q_i\}$ that determines Q, $p(Q) \leq \Sigma_i p(Q_i)$) to infer the existence of multi-relation values, while CIL and CIH derive data cardinality from the prices of `limit` and `count` queries, respectively. All of them have no assumptions on the prior knowledge and exhibit low computational complexity, making them practical. Besides, the last three algorithms have theoretical guarantees on inference precision in different cases.

Since typical leakage mitigation methods (e.g., data-oblivious processing [13]) exhibit inherent limitations (e.g., NP-hardness, high overpayment), we propose PerPricer, a perturbation-based pricing framework that computes prices over *perturbed data* to protect data information. The idea of PerPricer is to introduce additional qualified tuples (that satisfy query predicates) in data for pricing as perturbation to mislead attackers' inferences. It operates this goal via three stages, i.e., determining the number of extra qualified tuples, perturbed data generation, and price computation over perturbed data. PerPricer provides the theoretical guarantee against information leakage, while preserving original pricing time complexity. Besides, the overpayment of data buyers due to extra qualified tuples under PerPricer is bounded. The contributions of this paper are as follows.

- We formalize two novel attacks in query pricing, i.e., data existence and cardinality inference through query price, establishing the first systematic characterization of pricing-induced information leakage.
- We propose four information inference algorithms to derive sensitive information from prices of queries *without* any payment, having *near-optimal* inference accuracy.
- We develop a practical framework PerPricer to release the query price over *perturbed* data such that the real data information is protected with a theoretical guarantee, with a refund to the true price and a bounded buyer overpayment under PerPricer.
- Extensive experiments on four datasets verify the existence of information leakage in query pricing and the effectiveness and efficiency of the proposed algorithms.

The rest of the paper is organized as follows. We describe the preliminaries in Section II. Section III overviews the taxonomy of attacks that leak information. Sections IV–V detail the existence and cardinality inference algorithms, while Section VI formulates the masked-pricing mitigation problem and Section VII introduces PerPricer. The experimental results and our findings are reported in Section VIII. Section IX reviews the related work. We conclude this paper in Section X.

II. Preliminaries: Query Pricing

In this section, we describe query-based data markets and review existing query pricing methods.

In the query-based data market, the data seller has the on-sale database D with the relations $(R_1, R_2, \cdots, R_m)$. The database schema, the data size N_j of each relation R_j, and the domain of each attribute in R_j are known as public. The seller charges the data buyer according to the query Q rather than the full dataset. The price of a query Q is $p(Q)$ given a price function $p(\cdot)$. We focus on select-project-join-aggregate (SPJA) queries formed in SQL language.

Arbitrage-free pricing has been the key principle of data market design and widely studied in numerous works of data pricing [5], [16]–[18], [29]–[31], [34], [35], [37], [51]. In particular, the query pricing function $p(\cdot)$ exhibits l-arbitrage on the dataset D if there exists a set of queries $Q, Q_1, \cdots, Q_l$ such that $p(Q) > \sum_{i=1}^{l} p(Q_i)$ and the queries $Q_1, \cdots, Q_l$ determine Q on D. The determinacy relationship means that $Q(D)$ can be derived from $Q_1(D), \cdots, Q_l(D)$ without accessing the database D [40]. Definition 1 formulates the arbitrage-free property based on the query determinacy.

Definition 1: ***Arbitrage-free property***. A price function $p(Q)$ is arbitrage-free if whenever the query bundle $\mathcal{Q} = \{Q_1, \cdots, Q_l\}$ determines Q, there is $p(Q) \leq \sum_{Q_i \in \mathcal{Q}} p(Q_i)$.

The arbitrage-free property ensures that, data buyers cannot acquire the desired query for a *lower* price by combining other query results. Otherwise, data buyers would try their best for arbitrage, hindering the development of data markets.

Existing studies strive to achieve arbitrage-free pricing from two perspectives, i.e., usage-based and information-based pricing. *Usage-based pricing* (UBP) assigns the query price based on the underlying data that can *produce* the same query results, ensuring arbitrage-free pricing for SPJ queries [29]–[31], [37], [51]. Among UBP methods, view-based approaches (VBP) price each query as the total price of views that determine the query [29]–[31], whereas provenance-based methods (PBP) consider the price as the sum-up price of tuples contributing to the query answer [37], [51]. When multiple view/tuple sets can produce query results, the set with the minimal price is chosen to avoid arbitrage. PBP is the special case of VBP where each view corresponds to one tuple, which is much more efficient than VBP. The exponential complexity of VBP makes it impractical even on medium-size datasets (e.g., 1K tuples). As a result, we focus on the PBP approach as follows.

Information-based pricing (IBP) leverages the information that the query reveals to design arbitrage-free query price functions [16]–[18], [34], [35]. Specifically, the database is modeled as a *variable* with a set of possible values $\mathcal{S}_D$ and the real database instance D is in $\mathcal{S}_D$. The query answer $Q(D)$ can tell the information that any database D' with $Q(D') \neq Q(D)$ cannot be the real one. Such database information can be formulated as the *conflict* set $C_D(Q) = \{D' | D' \in \mathcal{S}_D \text{ and } Q(D') \neq Q(D)\}$. Further, arbitrage-free price functions are presented for SPJA queries based on $C_D(Q)$ [16]–[18], e.g., the total weight of possible databases

mid	name	country	rating
1	Inception	USA	8.8
2	Three idiots	India	8.4
3	Your name	Japen	8.4
4	E.T.	USA	7.9

(a) Movie relation

mid	week	country	grosses
1	1	USA	62,785,337
1	2	USA	42,725,012
2	1	India	8,665,678
3	1	USA	1,813,781

(b) Gross relation

Fig. 2. The on-sale database

D_1:	1	Inception	Europe	8.8
D_2:	1	Inception	USA	8.5
D_3:	2	Three idiots	India	7.0
D_4:	3	E.T.	Japan	8.4
	4	Your name	USA	7.9

(a) Possible databases on Movie

D_5:	1	1	USA	40,000,000
D_6:	1	2	India	8,665,678
	2	1	USA	42,725,012
D_7:	2	4	India	8,665,678
	3	1	Japan	1,813,781

(b) Possible databases on Gross

Fig. 3. The possible databases used in IBP for pricing

in $C_D(Q)$. Furthermore, each possible database D' contains *one or two* tuples different from the database D, to reduce the cost of comparing $Q(D')$ and $Q(D)$. In this case, all pricing functions under IBP are *data-dependent*. The pricing details are referred to the original paper [17], [18].

It is worth noting that, the price functions in IBP can be query answer-dependent or database instance-independent when possible databases are uniformly sampled from the distribution of the database value. However, as clarified in the original paper [17], [18], such a setting is impractical and meaningless. The query price can be as high as the price of a whole relation when only a smaller number of tuples are output, while the cost of price computation is expensive. Hence, we focus on the case where possible databases under IBP are generated from neighborhood databases, i.e., only one or two tuples in them are different from the real database.

Example 1: Fig. 2 depicts an on-sale movie database with two relations Movie and Gross. The data buyer Bob wants to purchase the query Q : `select * FROM Movie, Gross where Movie.mid = Gross.mid and week = 1 and grosses >` 10^7. The query answer $Q(D)$ can also be obtained by purchasing Q_1: `select * FROM Gross where week = 1 and grosses >` 10^7 and Q_2: `select * FROM Movie where mid in` O, where O contains the mid of tuples in $Q_1(D)$. Obviously, Q is determined by Q_1 and Q_2 and there should be $p(Q) \leq p(Q_1) + p(Q_2)$ to avoid arbitrage.

We illustrate the pricing process of existing studies for better understanding. Suppose the price of each tuple is 1 in provenance-based pricing (PBP). Then, the price of Q is 2 since the query answer $Q(D)$ is produced by the first tuples of Movie and Gross relations. Besides, Q_1 and Q_2 have the same price, i.e., 1 under PBP. Consider seven possible databases with the same weight $\frac{1}{7}$ in information-based pricing (IBP). As each possible database in $\mathcal{S}_D$ has one or two tuples taking different values from D, we plot the different tuples in each possible database in Fig. 3. Moreover, the price of Q is $\frac{4}{7}$ under IBP, where D_1, D_2, D_6, D_7 produce the query answer different from D. For instance, the query answer over D_6 is empty. Meanwhile, the price of Q_1 is $\frac{2}{7}$ since $Q_1(D) \neq Q_1(D_5)$ and $Q_1(D) \neq Q_1(D_6)$, while that of Q_2 is $\frac{2}{7}$ due to the different query answer of D_1 and D_2. As a result, both PBP and IBP derive the arbitrage-free price, satisfying $p(Q) \leq p(Q_1) + p(Q_2)$.

The above query pricing mechanisms exhibit inherent vulnerabilities due to their *data-dependent* nature, i.e., prices depend on both the query Q and the database D, enabling attackers to infer sensitive data information from the prices of queries. This critical security gap necessitates dual solutions: (i) systematic characterization of pricing-induced leakage, and (ii) development of leakage-resilient pricing frameworks. Our methodology progresses as follows. We first formalize the basic settings in pricing-induced leakage (i.e., threat model) and identify possible attacks (i.e., attack taxonomy). We further develop effective algorithms to infer sensitive information. Finally, we present the PerPricer framework to address the problem of leakage mitigation with the theoretical guarantee.

III. Overview of Information Leakage

This section formalizes the threat model in query pricing and establishes a taxonomy of price-driven inference attacks. The scope of the proposed information inference algorithms is also clarified.

A. Threat Model and Attack Taxonomy

The threat model comprises two entities, i.e., the *zero-knowledge* attacker and the query-based data trading system $\mathcal{M}$. First, the malicious attacker aims to learn the information of the on-sale database D without purchasing data. The trading system $\mathcal{M}$ employs existing arbitrage-free pricing schemes, i.e., provenance-based pricing (PBP) or information-based pricing (IBP) to price queries efficiently.

Assumption. The attacker is assumed to know only the database schema, attribute domain, and relation size of the public database before purchasing the query, while remaining unaware of the pricing logic and other information.

Attackers act like normal buyers in data markets, freely requesting query prices $p(Q)$ (as in Fig. 1). This indistinguishability between normal buyers and malicious attackers renders traditional access control mechanisms (like cryptographic protections [46], [54]) ineffective. The universal attack surface where any participant can become a potential attacker and substantially increases leakage risk at scale.

Following the existing works of information leakage [6], [13], we mainly study two categories of inference attacks, i.e., existence inference and cardinality inference. Fig. 4 shows the taxonomy of information leakage attacks. The *existence* inference attack extracts the information about whether a value v exists in the table of database D, while the *cardinality* inference attack tells how many times the value v appears in the table. Existence inference is also termed membership inference in some prior work [6], [48]. These primitive attacks enable composite variants through strategic query combinations, including max/min-value extraction, tuple reconstruction, and distribution inference.

In particular, the max/min value of an attribute can be inferred when (i) values higher/lower than v do not exist in the database and (ii) v exists in the database, so v is confirmed as the attribute extremum. Second, a tuple attribute leaks

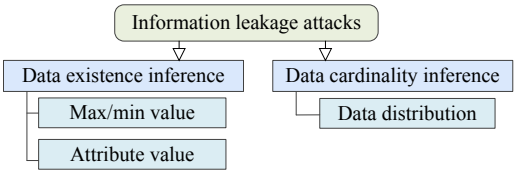

Fig. 4. Taxonomy of information leakage attacks

through repeated existence inference. Once ID x is confirmed, tOhe attacker probes for tuples with ID x and attribute value y across possible y until the full tuple is revealed. Even without explicit identifiers, they can combine several columns into a high-probability pseudo-identifier to recover the tuple. Moreover, the distribution of an attribute follows directly from multiple cardinality inference queries.

Overall, although the basic attacks (i.e., existence and cardinality inference) seem not to provide an attacker with significant information, they can form compounding attacks leaking actual values of tuples and attributes. Therefore, we mainly study the attacks of existence inference and cardinality inference, and four algorithms are proposed in Sections IV and V. Before introducing them, we first summarize the scope of these information inference algorithms.

B. Scope of Information Inference Algorithms

Specifically, We propose *Naive* and MI for existence inference, while CIL and CIH for cardinality inference. Their feasibility and costs are summarized below.

Table I summarizes feasibility across cases. $*/\times$ denote whether a theoretical accuracy guarantee is available or not. A single-relation value lies in one relation (e.g., "USA"), whereas a multi-relation value spans multiple relations, e.g., (mid=1, rating=8.8, gross=10^7). Our algorithms have accuracy guarantees in the marked cases. For cases without a formal guarantee, the empirical results still show high precision.

Furthermore, as described later, *Naive* and MI have constant time complexity, where the times of asking prices is no higher than the number of relations m. This is acceptable in practice, even when the data trading system $\mathcal{M}$ limits the requests per minute (RPM). The number of inquiring prices under CIL and CIH never exceeds $\log_2 N$ where N is the maximum possible cardinality. For instance, when the checked value v is on a single relation with size 10^9, its cardinality can be derived after asking about query prices 24 times.

Besides, these algorithms can be applied to the case where the size of each relation is unknown. For this case, CIL and CIH can take a large number (e.g., 10^9) as the input to derive the data cardinality, which represents the maximum possible cardinality. Meanwhile, *Naive* and MI are not affected by the unknown relation sizes.

Overall, the proposed inference algorithms remain practical across diverse scenarios while retaining theoretical guarantees on precision, efficiency, and minimal assumptions.

TABLE I
FEASIBILITY OF INFORMATION INFERENCE ALGORITHMS

Value type	Data existence		Data cardinality	
	PBP	IBP	PBP	IBP
Single-relation	*Naive**	*Naive*$^\times$	CIL*	CIH*
Multi-relation	MI*	MI*	CIL*	CIH$^\times$

IV. EXISTENCE INFERENCE ALGORITHMS

A. Naive method for the single-relation case

The goal of existence inference is to verify whether the value v exists in the database D. For v on the single relation, a *naive* approach is to construct a query Q_v that outputs some results if $v \in D$ and then check whether Q_v is priced as zero. The zero price indicates that v does not exist in the database. Otherwise, there is $v \in D$. Actually, *Naive* leverages such a typical idea, i.e., the empty query results should be priced as zero. *Naive* can infer the data existence for values on the single relation. It takes the value v to be checked and the data trading system $\mathcal{M}$ as inputs and outputs the existence information. It first constructs the query Q_v and asks for its price p. When $p = 0$, *Naive* returns the false existence, i.e., absent. Otherwise, it returns true.

The time complexity of *Naive* is $O(1)$. It can derive the *true* existence under provenance-based pricing (PBP), whose price depends on the query answer. However, *Naive* can not always obtain a true answer under information-based pricing (IBP), since the price relies on the query answer of both the on-sale database and possible databases. For instance, a value in the database can be free under IBP if it is the same across all these databases. Although *Naive* is not 100% accurate, it has high accuracy and F1 score in the evaluation.

B. MI algorithm for multiple-relation case

When the checked value v is on multiple relations, the existence of such a value can trigger the special pricing behavior for join queries. It is more accurate to leverage such pricing behavior for inference than directly applying *Naive* to multiple relations. For simplicity, we consider that v is a possible value over two relations T_1 and T_2, i.e., $v = (a, b, c)$ where $v_1 = (a, b)$ (or $v_2 = (b, c)$) is the possible value on T_1 (or T_2). The following analysis and algorithms can be applied to multiple relations. Specifically, consider three queries $Q_\bowtie$: `select * from` T_1, T_2 `where` T_1`.B = b and` T_1`.B =` T_2`.B and A = a and C = c`, Q_1: `select * from` T_1 `where A = a and B = b`, and Q_2: `select * from` T_2 `where B = b and C = c`. As v exists iff v_1 and v_2 do, the prices of the three queries have different relationships for $v \in D$ and $v \notin D$.

First, $v \in D$ tells that $v_1 \in D$ and $v_2 \in D$, and thus $Q_\bowtie$ is determined by Q_1 and Q_2. Further, there is $p(Q_\bowtie) = p(Q_1) + p(Q_2)$ under PBP and IBP. Second, for $v_1 \notin D$ (or $v_2 \notin D$), $Q_\bowtie$ is determined by Q_1 (or Q_2). The arbitrage-freeness ensures that $p(Q_\bowtie) = p(Q_1)$ (or $p(Q_\bowtie) = p(Q_2)$). For the case of $v_1 \notin D$ and $v_2 \notin D$, $p(Q_\bowtie) = 0$ holds for PBP and IBP. Therefore, whether $v = (a, b, c)$ belongs to the database can be inferred from the query prices.

Algorithm 1: The MI Algorithm

Input: the value $v = (a, b, c)$ to be checked and the data trading system $\mathcal{M}$
Output: the variables X indicating whether $v \in D$
1: ask $\mathcal{M}$ for the prices of $Q_1, Q_2, Q_{\bowtie}$ and obtain $p_1, p_2, p_{\bowtie}$ where Q_1 is `select * from` T_1 `where A = a and B = b`, Q_2 is `select * from` T_2 `where B = b and C = c`, and $Q_{\bowtie}$ is `select * from` T_1, T_2 `where` T_1`.B = b and` T_2`.B = b and A = a and C = c`
2: **if** $p_{\bowtie} = 0$ **then**
3: | **return** $X \longleftarrow 0$
4: **else if** $p_1 > 0$ and $p_2 > 0$ **then**
5: | **if** $p_{\bowtie} = p_1 + p_2$ **then**
6: | | **return** $X \longleftarrow 1$
7: | **else**
8: | | **return** $X \longleftarrow 0$
9: **return** $X \longleftarrow -1$

Algorithm 1 gives the pseudo-code of MI to infer the existence of a multi-relation value. It takes the value to be checked $v = (a, b, c)$ and the data trading system $\mathcal{M}$ as inputs, and outputs the indicating variable X. $X = 1$ (or 0) means v is in the database (or not), while $X = -1$ tells that whether $v \in D$ is uncertain. First, the algorithms asks $\mathcal{M}$ for the prices $p_1, p_2, p_{\bowtie}$ of these three queries Q_1, Q_2, and $Q_{\bowtie}$. When $Q_{\bowtie}$ is free, one can directly return $X = 0$ (lines 2-3). When p_1 and p_2 are positive, MI updates and returns $X = 1$ if $p_{\bowtie} = p_1 + p_2$ and otherwise returns $X = 0$ (lines 4-8). For the cases of $p_1 = 0$ (or $p_2 = 0$), MI returns $X = -1$, since it can not distinguish from $p_{\bowtie} = p_2$ (or $p_{\bowtie} = p_1$) and $p_{\bowtie} = p_1 + p_2$.

The time complexity of the MI algorithm is $O(1)$, since there are several price comparisons for existence inference. Moreover, the correctness of Algorithm 1 is ensured by Theorem 1. It can be proved by analyzing the pricing behavior of PBP and IBP. The proof is provided in the supplementary material due to space constraints.

Theorem 1: The certain existence derived in Algorithm 1 is consistent with the ground truth.

Example 2: Suppose the attacker wants to derive the existence of the value v = (mid = 1, rating = 8.5, country = USA) and there is $v \in D$. To derive the existence information, the attacker first constructs the queries Q_1: `select * from Movie where mid = 1 and rating = 8.5`, Q_2: `select * from Gross where mid = 1 and country = USA`, and $Q_{\bowtie}$: `select * from Movie, Gross where Movie.mid = 1 and Gross.mid = 1 and rating = 8.5 and country = 'USA'`. When the pricing method is PBP, the prices of these three queries are 0, 2, and 0, respectively. When the pricing method is IBP, the prices of these queries are $\frac{1}{7}$, $\frac{2}{7}$, and $\frac{1}{7}$, respectively. The prices of Q_1 and $Q_{\bowtie}$ are based on the possible database D_2, while that of Q_2 is based on D_5 and D_6. In these cases, MI can derive that $X = 0$, i.e., $v \notin D$, which is the same as the ground truth.

Overall, MI leverages the arbitrage-free property to reliably infer data existence in $O(1)$ time, which is independent of the pricing mechanism in the trading system. The uncertain existence can be further resolved by iteratively applying MI on the uncertain value and other verified existent values.

V. Cardinality Inference Algorithms

In this section, we analyze how an attacker extracts data cardinality from prices of parametric queries and propose two cardinality inference algorithms designed for PBP and IBP.

Cardinality inference aims to obtain the number of times the value v appears in the database. Although a larger number of returned tuples may result in a higher price, it is still difficult to directly infer the cardinality of the query answer from the price. This is because, the price is not linear with the query cardinality, where each tuple can have a different price under provenance-based pricing (PBP) and information-based pricing (IBP). Therefore, it motivates us to try to control the cardinality of the query and observe the varying prices, in order to infer the actual query cardinality. In the following, we propose two cardinality inference methods that leverage the `limit` and `having` clauses to achieve the above idea on PBP and IBP, respectively.

A. Inference based on `LIMIT` clause

Specifically, the `limit` K clause is the typical way for database users to enable precise control over the size of the result set, i.e., at most K results are retrieved. The pricing approaches would derive different prices when varying K. Let k be the actual cardinality of the query Q, Q_K be the query that adds the `limit` K clause on the top of Q, and p_Q and p_K be the price of Q and Q_K, respectively. Obviously, Q_K has the same answer as Q when $K = k$. Moreover, the price p_K under PBP equals p_Q when $K \geq k$, since PBP considers the used tuples for pricing. IBP is possible to price Q_K the same as Q for $K \neq k$, depending on the specific possible databases. Therefore, when the pricing approach is PBP, one can vary the parameter K of Q_K to find the first point with $p_K = p_Q$ for cardinality inference.

Algorithm 2 illustrates the pseudo-code of CIL algorithm for cardinality inference under PBP. It takes the value a to be checked, the size N_T of the relation T, and the data trading system $\mathcal{M}$ as inputs, and outputs the inferred occurrence frequency of a in T. First, it asks the system $\mathcal{M}$ to derive the price p of the query Q. If the price p is zero, it directly purchases the query and returns the actual query cardinality. Then, CIL starts to perform *binary search* on the range $[0, N_T]$ to find the *first* element K with the query price $p_K = p$. The left and right points (i.e., the minimal and maximal possible cardinality) are initialized as 0 and N_T, respectively. At each iteration, K is updated as the mid-point of $[left, right]$ (line 6), and the price p_K of Q_K is obtained. If p_K is no less than p, the right point is updated as K. Otherwise, CIL sets the left point as $K + 1$. The iteration ends when the left point equals or is higher than the right point. Finally, the left point is the first element with $p_K = p$ for $K = k$ and CIL returns it.

The time complexity of the CIL algorithm is $O(\log N_T)$ due to the binary search on the range $[0, N_T]$ where N_T is the size of the relation T. Theorem 2 confirms the effectiveness of CIL, whose proof is provided in the supplementary material due to space constraints.

Algorithm 2: The CIL Algorithm

Input: the value a to be checked, the size N_T of the relation T, and the data trading system $\mathcal{M}$
Output: the inferred occurrence frequency of a in T

```
ask M for the price p of the query Q: select * from T where A = a
if p = 0 then
    purchase Q(D); return |Q(D)|
left ⟵ 0; right ⟵ N_T
while left < right do
    K ⟵ ⌊(left+right)/2⌋
    ask M to derive the price p_K of the query Q_K: select * from T where A = a limit K
    if p_K ≥ p then
        right ⟵ K
    else
        left ⟵ K + 1
return left
```

Theorem 2: The cardinality inferred by CIL equals the real one k for the provenance-based pricing approach PBP.

Example 3: Consider inferring the number of movies with rating < 8.5 (query Q: `select * from Movie where rating<8.5`). Following CIL, we first price Q under provenance-based pricing (PBP) and obtain $p = 3$, so we initialize $left = 0$ and $right = 4$ (relation size). Each iteration sets $K = \lfloor (left + right)/2 \rfloor$ and queries Q_k: `select * from Movie where rating<7.5 limit` K to get p_k. If $p_k < p$, move $left$ to $K + 1$, otherwise set $right$ as K. In this example K takes 2 then 3, yielding $p_k = 2$ and $p_k = 3$, so $left$ is set to 3 and $right$ shrinks to 3. Then the search is ending. The algorithm returns $left = 3$, matching the true cardinality.

Extension to the multi-relation case. CIL can be directly used to infer the cardinality of any multi-relation value v. The only difference is that CIL uses the maximum possible cardinality $\prod_{T_i \in R_Q} N_i$ as the input where R_Q contains the relations that v locates.

Besides, CIL is also applicable for information-based pricing (IBP) but has a high inference error. Specifically, for the conflict database D_1 of Q with $Q(D) \neq Q(D_1)$ at the j-th record, it is used for pricing Q_K when $K = j$. In light of this, the inferred cardinality is the maximal j among all conflict databases of Q. Hence, it is possible to infer cardinality far from the real one. Actually, such an error comes from that, the price of the `limit` queries under IBP relies on the order of different tuples in conflict databases. To address the issue, we propose a new method for accurate cardinality inference under IBP, based on the prices of queries with `having count(*)` $= K$ clause. The main idea is that the answer of *count* query is unaffected by the order of tuples.

B. Inference based on `HAVING` clause

Formally, let Q_K be `select count(*) from` T `where A = a having count(*)` $= K$, k be the count of the value a in the relation T, and p be the price of the query `select count(*) from` T `where A = a`. Table II illustrates the behavior of IBP on pricing Q_K. Recall that IBP employs possible databases with one or two tuples different from the real database. Therefore, in these databases, the occurrence frequency of the value a varies from $k - 2$ to $k + 2$. Let $\mathcal{N}_j$ contain the possible databases $\mathcal{D}$ where the value a appears $k + j$ times in $\mathcal{D}$. Moreover, given the parameter $K < k - 2$ or $K > k + 2$, the query Q_K has the empty result on D and all possible databases, leading to the zero price. For $K = k - 2$, only the databases in $\mathcal{N}_{-2}$ have a non-empty answer, where Q_K is priced based on $\mathcal{N}_{-2}$. Similarly, when $K = k - 1, k + 1, k + 2$, Q_K is priced based on $\mathcal{N}_{-1}$, $\mathcal{N}_1$, $\mathcal{N}_2$ respectively. Meanwhile, the price of Q_K with $K = k$ (denoted as p) is the highest, since all databases in $\mathcal{N}_{-2} \bigcup \mathcal{N}_{-1} \bigcup \mathcal{N}_1 \bigcup \mathcal{N}_2$ produce query answers different from the real database D.

TABLE II
PRICING BEHAVIOR OF IBP FOR CARDINALITY INFERENCE

The value of K	Used possible databases	Price
$K < k - 2$ or $K > k + 2$	$\varnothing$	0
$K = k - 2, k - 1, k + 1, k + 2$	$\mathcal{N}_{-2}, \mathcal{N}_{-1}, \mathcal{N}_1, \mathcal{N}_2$	$\leq p$
$K = k$	$\mathcal{N}_{-2} \bigcup \mathcal{N}_{-1} \bigcup \mathcal{N}_1 \bigcup \mathcal{N}_2$	p

Overall, the query Q_K shares the same price as Q under the following cases. First, when there are at least two non-empty sets among $\mathcal{N}_{-2}, \mathcal{N}_{-1}, \mathcal{N}_1$, and $\mathcal{N}_2$, Q_K is priced the same as Q when $K = k$. Second, when there is only one non-empty set $\mathcal{N}_j$ in the four sets, the prices of Q_K and Q are the same for $K = k$ and $K = k + j$. For the case of $\mathcal{N}_{-2} \bigcup \mathcal{N}_{-1} \bigcup \mathcal{N}_1 \bigcup \mathcal{N}_2 = \varnothing$, the price of Q_K is always zero. Based on the observed pricing behavior, it is easy to derive the actual count of a value with a very tiny error (i.e., 1) by asking the price of Q_K for $K = 0, \cdots, N_T$, where N_T is the relation size. The time cost can be reduced in the logarithmic level by considering the price of Q_K^h: `select count(*) from` T `where A = a having count(*)` $\geq K$, whose price first equals p and then decreases as zero. The non-increasing property of the price sequence can be used to find the first element K such that Q_K^h is priced as zero, where K is close to real cardinality k.

Algorithm 3 illustrates the pseudo-code of CIH algorithm for cardinality inference under IBP. It takes the value a to be checked, the size N_T of the relation T, and the data trading system $\mathcal{M}$ as inputs, and outputs the inferred occurrence frequency of a in T. First, it asks the system $\mathcal{M}$ to derive the price p of the query Q. If the price p is zero, the query answer is purchased and the actual occurrence frequency is returned. Otherwise, CIH employs the binary search to find the first element $c \in [0, N_T]$ such that Q_c^h is priced as zero. Next, it initializes k and n as zero and derives the value of k based on the pricing behavior in Table II. CIH asks the price of Q_K for $K = \max\{c-5, 0\}, \cdots, \min\{N_T, c+1\}$ and updates k as the average of K where Q_K shares the same price as Q. Finally, CIH rounds k down and returns $\lfloor k \rfloor$.

The CIH algorithm requires the binary search in the range $[0, N_T]$ for cardinality inference, which has $O(\log N_T)$ time complexity. N_T is the size of the relation T. Theorem 3 guarantees that CIH achieves reliable inference under information-based pricing (IBP), with a mean absolute error (MAE) of one.

Theorem 3: Let k be the actual occurrence frequency of

Algorithm 3: The CIH Algorithm

Input: the value a to be checked, the size N_T of the relation T, and the data trading system $\mathcal{M}$
Output: the inferred occurrence frequency of a in T
1: ask $\mathcal{M}$ to derive the price p of the query Q: `select count(*) from` T `where A = a`
2: **if** $p = 0$ **then**
3: purchase $Q(D)$ and set k as $\{Q(D)\}$; **return** k
4: use binary search to find the first element $c \in [0, N_T]$ such that Q_c^h is priced as zero, where Q_c^h is `select count(*) from` T `where A = a having count(*)` $\geq c$
5: $k \longleftarrow 0, n \longleftarrow 0$
6: **for** $K = \max\{c-5, 0\}$ **to** $K = \min\{c+1, N_T\}$ **do**
7: ask $\mathcal{M}$ to derive the price p_K of the query Q_K: `select count(*) from` T `where A = a having count(*)` $= K$
8: **if** $p_K = p$ **then**
9: $k \longleftarrow \frac{n}{n+1} \cdot k + \frac{1}{n+1} \cdot K$
10: **return** $\lfloor k \rfloor$

value a and $\hat{k}$ be the inferred one from CIH. $|k - \hat{k}| \leq 1$ holds $\forall k \in [0, N_T]$, i.e., the MAE of CIH is at most one.

Proof. First, the value c derived in line 4 of CIH can be proved to satisfy $k + 1 \leq c \leq k + 3$. In particular, the price of the query Q_c^h has the following features. For $c \leq k$, Q_c^h is priced based on $\mathcal{N}_{-2} \bigcup \mathcal{N}_{-1} \bigcup \mathcal{N}_1 \bigcup \mathcal{N}_2$, which have query answers different from the real one. Hence, the price of Q_c^h equals the price p of the query Q: `select count(*) from` T `where A = a`. Moreover, when c becomes $k+1$, there are $Q_c^h(D) = \varnothing$ and $Q_K(D') \neq \varnothing$ for $D' \in \mathcal{N}_1 \bigcup \mathcal{N}_2$. The price of $Q_c^h(D)$ can be zero for $\mathcal{N}_1 \bigcup \mathcal{N}_2 = \varnothing$ or positive otherwise. In addition, for $c = k + 3$, Q_c^h certainly has the zero price since all possible databases and the real one have the empty answer. Therefore, the first element c with the zero price meets $k + 1 \leq c \leq k + 3$.

Second, based on $k + 1 \leq c \leq k + 3$, there are $c - 5 \leq k - 2 \leq c - 3$ and $c - 1 \leq k + 2 \leq c + 1$. Hence, CIH checks the price of Q_K for $K = c-5, \cdots, c+1$, to further derive the estimation of the real k. As mentioned before, CIH may find one element $K = k$ or two elements $K = k$ and $K = k + j$ such that the price of Q_K is p, where j can be $-2, -1, 0, 1, 2$. As the final inference in CIH is the floor of the average of these elements, it can take the values k, $k-1$, and $k+1$. As a result, the mean absolute error of CIH on cardinality inference is at most one. The proof is complete. □

It is noteworthy that CIH can be iteratively applied to enhance the inference precision. When the actual count k_i of one value a_i can be reliably determined (i.e., only one K meets the condition in line 8 of CIH), one can employ CIH to infer the total count $k_i + k_j$ of a_i and a_j. When $k_i + k_j$ is reliable, the hierarchical inference can remove the slight error of inferring the count of a_j. In addition, one can also leverage other clauses to infer the data cardinality, whose idea is similar to that using `having`. For instance, the `case when` clause can also control outputs based on the cardinality size.

Extension to the multi-relation case. It incurs high error if directly applying CIH to any multi-relation value v, since the cardinality of v over all possible databases is not in $[k-2, k+2]$ where k is v's real cardinality. This is because, one/two different tuples can produce zero or many results after the join operation. Despite this, one can decompose v into multiple single-relation values $v_1, \cdots, v_l$, and employ CIH to infer the cardinality of v_i separately. Let $\hat{k}_i$ be the inferred cardinality of v_i, one can further infer the cardinality of v as $\hat{k} = \prod_{i=1}^{l} \hat{k}_i$. In the worst case, the inference error $|k - \hat{k}|$ can be pretty high. Thus, it is suggested to use the hierarchical inference to derive reliable $\hat{k}_i$ to reduce the error.

VI. Formulation of Leakage Mitigation Problem with Masked Pricing

In this section, we first present typical approaches to mitigate information leakage, then analyze their limitations in practice, and finally formulate the problem of leakage mitigation based on masked price.

A. Typical Leakage Mitigation Approaches

As described in Section III, there are multiple information leakage types enabling attackers to infer database content without prior knowledge or financial cost. This necessitates developing mitigation strategies that effectively *prevent* leakage while *minimizing* impacts on system participants. Many leakage mitigation strategies introduce overpayment to data buyers, which should be minimized to protect their interests. Moreover, typical mitigation methods can be designed *independent* of the underlying leakage mechanisms.

Specifically, following the ideas of prior research, various strategies can be used to mitigate information leakage [6], [13], [34], including query restriction, data-oblivious pricing, and delayed pricing. Query restriction *prohibits* queries that may leak sensitive information, while data-oblivious pricing *decouples* query prices from underlying data. Delayed pricing releases the query price *concurrently* with the query answer, ensuring the price remains undisclosed beforehand. These three approaches can fully avoid information leakage, since the correlation between price and data is removed or hidden.

However, these strategies have significant limitations in the context of data trading, making them impractical. The selection and join queries become unavailable under the query restriction, thereby invalidating the operational foundation of query-based markets. Moreover, it is NP-hard to compute the data-oblivious (i.e., database instance-independent) price [17]. Under delayed pricing, normal data buyers cannot know the payment in advance and *must* purchase the whole database if they want to know the query price. It seriously reducing their willingness-to-pay, leading to numerous overpayment.

In addition, the differential privacy [19], [20] (DP) is a rigorous mathematical framework designed to provide strong privacy guarantees when analyzing or releasing data. However, it has some drawbacks when employing DP in query pricing. On the one hand, DP focuses more on the results of queries rather than the prices of queries. Even if we use DP on data pricing, it will add the Laplace noise into the price, making same query have different prices. The *unstable* pricing behavior would reduce the experience of data buyers. To mask the '0 vs α' price gap, a DP machanism on prices must add noise at scale $b = \Delta f / \epsilon = \Omega(\alpha)$, which directly shows

as price jitter, where α is the minimum price gap between 'no result' and 'result'. Here, Δf is the sensitivity of the pricing function, and ϵ is the privacy budget. With privacy budget $\epsilon = 1$, the price jitter $b = 1$, i.e., one unit price jitter, which is unacceptable to data buyer in practice. On the other hand, DP will clip and project to attempt to ensure the arbitrage-free property, introducing downward bias that breaks the monotonicity of query inclusion relations. Independent noise also violates additivity, creating arbitrage opportunities.

Overall, it is impractical to mitigate the information leakage by *fully disabling* the correlation between query price and query answer. While releasing *real* query prices inevitably leaks some data information, it motivates us to seek the approach of publishing the *masked* price such that (i) attackers cannot derive accurate data information and (ii) the drawbacks of typical mitigation methods are addressed. In the following, we formulate the problem of generating the masked price and propose a framework to achieve the goals.

B. Masked Pricing for Leakage Mitigation

Now we focus on releasing the *masked* query price to mitigate the information leakage while minimizing the impacts on data sellers and buyers. It thus puts several requirements on the masked price. On the one hand, the masked price should be *no* less than the real one. Otherwise, the data broker would suffer a revenue loss once the masked price is paid. On the other hand, the overpayment due to the masked price should be *bounded* and can be *refunded* to the data buyers. In this way, data buyers are aware of the maximum overpayment and finally pay for the actual price. Also, the masked price should work on all existing query pricing algorithms and can be derived as quickly as possible. Considering these aspects together, our studied problem is formulated in Definition 2.

Definition 2: ***Problem statement***. Suppose there is an on-sale dataset D with relations $(R_1, R_2, \cdots, R_m)$ and an arbitrage-free query pricing function $p(\cdot)$. The studied problem is to design a masked pricing mechanism $\mathcal{W}$ that derives the masked price $\hat{p}_Q$ for each buyer-specified query Q with the following characteristics:

- *Generality*. $\mathcal{W}$ should be applicable for existing query pricing works PBP and IBP.
- *Efficiency*. The masked price $\hat{p}_Q$ should be obtained in polynomial time.
- *Bounded overpayment*. The overpayment $\hat{p}_Q - p(Q)$ should be bounded by a positive constant C.
- *Theoretical guarantee*. The attacker cannot derive or derive wrong data information from masked prices.

In masked pricing, the core operation is to generate the masked price for each query efficiently and effectively. As information leakage arises from the *varied* pricing patterns under different data existence or cardinality, there are two natural ways to block information leakage. The first one is to *disable* the varied pricing patterns to prevent information inference, while the second one is retaining these pricing patterns while disclosing prices based on *synthetic* data to *obfuscate* attackers.

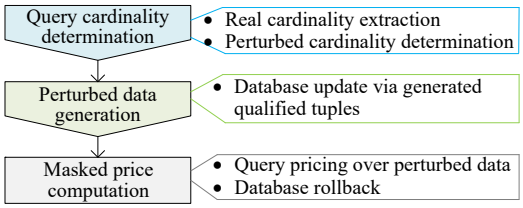

Fig. 5. The PerPricer framework

However, directly disabling the special price patterns results in a high and uncontrollable overpayment. For instance, one can release the prices of join queries as the summation of multiple single-relation queries, where MI algorithm becomes invalid. However, this strategy performs like delayed pricing when handling normal join queries. When the actual price of a join query is small (as a few tuples can be joined), the released price can be pretty high, where the overpayment is bounded by the total price of all involved relations. Therefore, although this masked price generation strategy is easy to implement, it is not useful in the context of query pricing where a small price increase may lead to customers leaving.

Therefore, we adopt the second approach, i.e., pricing over *synthetic* data to mislead the attackers. The core challenge is how to ensure the leaked information over synthetic data *different* from that on real data. For example, if a value is absent/present in both real and synthetic data, attackers still infer the real data existence. It is nontrivial to guarantee that one synthetic dataset has different data existence and cardinality information from the real one in all cases. To address this, we develop the framework PerPricer generating query-specific extra data for masked pricing. This prevents attackers from identifying nonexistent values or deriving accurate cardinality based on the information inference algorithms. PerPricer remains *compatible* with existing (and future) query pricing systems by preserving their internal logic.

VII. Pricing over Perturbed Data

In this section, we propose the PerPricer framework that releases the query price using perturbed data to reduce information leakage with bounded overpayment.

Recall that attackers infer data existence and cardinality from the following price patterns. First, nonexistent values v may yield *free* queries or the price of the join query *deviating* from summed individual prices. Second, the parametric queries with `limit` K or `having count(*)` $= K$ clause have the *peak* price when K nears the real cardinality k. To reduce the risk of information leakage, perturbed data must include *nonexistent* v to disrupt the first pattern and inflate *qualified* tuples (that satisfy query predicates) beyond real counts to obscure the second pattern. As a result, PerPricer generates such data for each coming query and releases the prices masking true database states.

Fig. 5 plots the overview of PerPricer framework that prices over the perturbed data to mitigate the information leakage with the theoretical guarantee. It consists of three parts, i.e., query cardinality determination that obtains the number of

extra qualified tuples, perturbed data generation that updates the underlying database D with generated qualified tuples, and masked price computation that employs the pricing system on the updated database and rolls back the database. In this way, the attacker can derive that any value exists in the database, while the inferred cardinality must differ the real one.

Specifically, we employ the *hash function* $H(k)$ to determine the number of extra generated tuples on each relation, where k is the actual number of qualified tuples meeting query predicates in this relation. The *fixed* and *random* strategies are not considered. On the one hand, the fixed value can be easily inferred when the attacker has already known the cardinality of a query, e.g., via purchasing a cheap query. On the other hand, the random value can lead to pricing inconsistency, i.e., the identical query can have different prices, potentially triggering price discrimination concerns [27], [57].

Moreover, for a join $Q_{\bowtie}$ over multiple relations, we must obtain per-relation qualified counts $k_1, \ldots, k_\ell$ to compute $H(k_1), \ldots, H(k_\ell)$. If any relation has zero qualified tuples, the join is empty and these counts are unavailable. We therefore insert a synthetic tuple t that satisfies $Q_{\bowtie}$'s predicates into all involved relations to make the join nonempty, then derive $k_1, \ldots, k_\ell$ by scanning $Q_{\bowtie} D \cup t)$ and record the qualified tuple IDs for the subsequent update.

Given the number of extra qualified tuples on each relation, the next step is to generate and utilize them to replace the tuples $\hat{t}$ in D that *break* the query predicates. Under information-based pricing (IBP), the tuples $\hat{t}$ should also take different values over D and one possible database. Formally, let H_i be the ID of qualified tuples of the join query $Q_{\bowtie}$ on the relation R_i, the tuple $\hat{t}$ to be replaced under IBP satisfies that $\hat{t} \notin H_i$ and there exists at least one possible database $D' \in \mathcal{S}_D$ where $D'[\hat{t}] \neq D[\hat{t}]$. Actually, these tuples have a positive price under IBP. If updating other tuples, the query price under IBP would not change after the database update, where information leakage still exists. Finally, the perturbed price can be derived by employing the pricing system on the updated database, while the database is rolled back after pricing.

Algorithm 4 depicts the procedure of generating the masked price over the perturbed data. It takes the original database D, the hash function $H(\cdot)$, the data trading system $\mathcal{M}$, and the query Q on relations $R_1, \cdots, R_l$ to be priced as inputs. It outputs the perturbed query price $\hat{p}_Q$. First, PerPricer generates one (joined) tuple that satisfies the query predicates of Q and the corresponding tuples $t_1, \cdots, t_l$ are inserted into the database (line 1). Then, the set W is initialized as the query answer that asks for the ID of qualified tuples on each involved relation. Next, for each relation involved in Q, PerPricer starts to extract the number of qualified tuples, determine the number of extra generated tuples, and update the database (lines 3-9). In particular, the impact of the inserted tuple t_i is removed (line 4). H_i is updated to record the ID of qualified tuples and k is set as $|H_i|$ (line 5). PerPricer then employs the value of t_i to update the first $H(k)$ tuples that break the query predicates and have a positive price (lines 6-9). After that, PerPricer derives the perturbed price $\hat{p}_Q$ based on the pricing system $\mathcal{M}$

Algorithm 4: The PerPricer Algorithm

Input: the original database D, the hash function $H(\cdot)$, the pricing system $\mathcal{M}$, and the query Q on $R_1 \cdots, R_l$ relations
Output: the perturbed price $\hat{p}_Q$
1: generate one (joined) tuple t satisfying the predicates of Q and insert the corresponding tuples $t_1, \cdots, t_l$ into D
2: initialize the set W as the answer of the query `select` $R_1.ID$, $\cdots$, $R_l.ID$ `from` $R_1, \cdots, R_l$ `where predicates of` Q
3: **foreach** R_i involved in Q **do**
4: remove the ID of t_i from W and delete t_i from R_i
5: $H_i \longleftarrow$ the distinct ID set of W on R_i, $k_i \longleftarrow |H_i|$
6: **if** $\mathcal{M}$ uses PBP for pricing **then**
7: employ the value of t_i to update the first $H(k)$ tuples in R_i whose IDs are not in H_i
8: **else**
9: employ the value of t_i to update the first $H(k)$ tuples in R_i whose IDs are not in H_i and have a positive price
10: employ $\mathcal{M}$ to derive the price $\hat{p}_Q$ of Q on the modified D
11: remove the updates on the database D
12: **return** $\hat{p}_Q$

and the modified database D (line 10). Finally, the updates on D are removed and the perturbed price is returned.

The time cost of PerPricer mainly consists of two parts, executing and pricing the query Q as well as the database update. As the query execution and database update can be done in *linear* time, the time complexity of PerPricer is the same as that of query pricing.

Example 4: Consider the existence inference attack in Example 2, the attacker leverages the prices of Q_1, Q_2, and $Q_{\bowtie}$ to infer the existence of v_1 = (mid = 1, rating = 8.5) and v_2 = (mid = 1, country = USA). In particular, the prices of these three queries under PBP are 0, 2, and 0, respectively, while they are priced as $\frac{1}{7}, \frac{2}{7}$ and $\frac{1}{7}$ under IBP.

Moreover, the masked prices of the three queries under PerPricer can be derived as follows. For the join query $Q_{\bowtie}$, PerPricer generates one joined tuple $t = v_1 \bowtie v_2$, and inserts v_1 and v_2 into the Movie and Gross relation, respectively. The values v_1 and v_2 on other attributes can be randomly generated. Moreover, suppose there is an extra aID column as the identifier in each relation, the query is constructed as `select m.aID, g.aID from Movie m, Gross g where m.mid = g.mid and rating = 8.5 and country = USA`, and the query answer W can be obtained as $\{(-1,1),(-1,2),(-1,-1)\}$ where the -1 corresponds to the generated data. Hence, one can derive that $k_1 = 0$ and $k_2 = 2$. Let the hash function be $H(k) = k \mod 2 + 1$, there are $H(0) = 1$ and $H(2) = 1$. PerPricer thus updates the first tuple in Movie relation with the value of (mid = 1, rating = 8.5), while the third tuple in Gross is updated with (mid = 1, country = USA). Finally, PerPricer computes the masked price of $Q_{\bowtie}$ as 4 under PBP. Meanwhile, the masked price of $Q_{\bowtie}$ is $\frac{4}{7}$ under IBP, due to the possible databases D_1, D_5, D_6 and D_7. Similarly, the masked prices of Q_1 and Q_2 are 1 and 3 under PBP, while those are $\frac{1}{7}$ and $\frac{3}{7}$ under IBP. As a result, given the masked prices of queries, the attacker can derive wrong data existence, i.e., both v_1 and v_2 are in the database.

Actually, the masked price $\hat{p}_Q$ of any query Q derived in PerPricer comprises two parts, i.e., the price of k_i qualified

tuples and that of extra generated $H(k_i)$ tuples on each relation R_i. In view of this, the upper bound of $\hat{p}_Q$ can be derived in Lemma 1, while the risk of information leakage is reduced with theoretical guarantee in Theorem 4. Moreover, the overpayment of data buyers under PerPricer can be directly obtained based on Lemma 1. PerPricer maintains compatibility with existing PerPricer pricing schemes (e.g., PBP or IBP) by preserving their core logic. Overall, PerPricer can derive the masked price of any query over the perturbed data, achieving the four characteristics in Definition 2.

Lemma 1: Let $p(Q)$ be the actual price of the query Q and $\hat{p}_Q$ be the corresponding price derived in PerPricer. When Q is on a single relation R_i and k tuples satisfy the query predicates, there is $\hat{p}_Q \leq p(Q) + MP^i_{H(k)}$. When Q is on multiple relations $R_1, \cdots, R_l$ and k_i $(i = 1, \cdots, l)$ tuples in R_i meet the query predicates, there is $\hat{p}_Q \leq \sum_{i=1}^{l} MP^i_{H(k_i)+k_i}$. MP^i_j is the maximal total price of j tuples in R_i.

Proof. Let MP^i_j be the maximal total price of j tuples in R_i. First, when the query Q is on a single-relation and k tuples satisfy the query predicates, PerPricer actually injects another $H(k)$ tuples into the database for pricing. Thus, the derived price $\hat{p}_Q$ is never higher than the total price of k and $H(k)$ tuples, i.e., $\hat{p}_Q \leq p(Q) + MP^i_{H(k)}$. For the query Q on multiple relations, PerPricer derives the price as the total price of $k_1 + H(k_1)$ tuples on $R_1, \cdots$, and $k_l + H(k_l)$ tuples on R_l, i.e., $\hat{p}_Q \leq \sum_{i=1}^{l} MP^i_{H(k_i)+k_i}$. Note that, $\hat{p}_Q \leq p(Q) + \sum_{i=1}^{l} MP^i_{H(k_i)}$ does not hold for empty join query Q, since $p(Q)$ can be less than the total price of $k_1, \cdots, k_l$ tuples, as proved in Theorem 1. The proof is complete. □

Theorem 4: Under PerPricer, CIH and CIL always infer that every value v exists and report cardinality $k + H(k)$.

Proof. Recall that MI infers existence from the price of a join query. Under PerPricer, the masked join price equals the sum of the masked prices on the single relations, regardless of whether v exists. Thus MI always concludes that v exists. Meanwhile, PerPricerupdates the database so that the qualified-count becomes $k + H(k)$. The attacker therefore infers the wrong cardinality. The proof is complete. □

Discussion. PerPricer discloses the masked price computed on the original data and generated data satisfying the query conditions, providing guarantees on defending against the proposed attacks. The pricing perturbations under PerPricer can exhibit non-arbitrary characteristics, ensuring the stable masked price of each query. It is superior to typical mitigation methods in Section VI-A, e.g., differential privacy suffers from unstable pricing and residual information leakage while other methods incur prohibitive computational costs or overpayment. PerPricer demonstrates adaptability for future query pricing works due to its independence from underlying pricing mechanisms. In addition, the paradigm of releasing prices over *synthetic* data provides a viable strategy to mitigate *potential* information leakage through query prices in the future.

VIII. Experiments

In this section, we verify the existence of possible information leakage and evaluate our proposed algorithms. All algorithms were conducted on an Intel Core 2.90GHz Server with 512GB RAM, running Ubuntu 20.04 system. We use two real-world datasets *Walmart* [21], *Employment* [14], and two benchmark datasets *TPC-H* [52], *SSB* [42], as descried in Table III. Specifically, *Walmart* and *Employment* are sourced from the AWS data marketplace [1]. *Walmart* comprises a single table that encapsulates detailed information (e.g., the location and open time) about Walmart within the United States. *Employment* includes a table that delineates employment statistics across various industries in the United States. *TPC-H* and *SSB* are popular benchmarks for performance metrics on large-scale systems, which have multiple relations and occupy about 1GB of memory. To evaluate the performance of the proposed attack algorithms and mitigation methods, we randomly generate 10K values on *Walmart* and *Employment* as well as 100K values on *TPC-H* and *SSB*, whose existence and cardinality are tested. The test values on *Walmart* and *Employment* are on the single relation. In contrast, *TPC-H* and *SSB* allocate equal proportions of test values to single-relation and multiple-relation cases.

TABLE III
Dataset statistics

Dataset	# Rel.	# Tuples	# Attri.	# Tests
Walmart	1	4,636	12	10,000
Employment	1	2,624	8	10,000
TPC-H	8	8,661,245	61	100,000
SSB	5	6,235,730	56	100,000

We simulate a query-based data trading market with state-of-the-art pricing: provenance-based pricing (PBP) and information-based pricing (IBP) [17], [18], [37], [51]. PBP prices a query as the minimal total price of *tuples* contributing to its answer [37], [51], while IBP prices SPJ and aggregation queries from database-level information [17], [18]. We implement all methods in Python over MySQL 8.0. Following the original IBP settings, we use 1K possible databases for *Walmart* and *Employment*, and 1M for the other datasets.

To evaluate the proposed attack algorithms, we compute the accuracy and F1 score between the inferred existence and the actual one, while the mean absolute error (MAE) and mean square error (MSE) are utilized to assess cardinality inference. We use earth mover's distance (EMD) to measure the difference between the inferred data distribution and the real one. The CPU time is measured to evaluate the efficiency. These metrics are also used to verify the effectiveness of mitigation methods, while the *overpayment* is quantified to illustrate the effect of the mitigation on data buyers.

In the experiments, we evaluate the effectiveness of the proposed four algorithms on inferring data information, including *Naive*, MI, CIL, and CIH. Moreover, we compare the proposed PerPricer with the three baselines. The first one is to add a fixed price over the real price (termed as *FP*), while the second one is the delayed pricing (termed as *DP*) that publishes the maximum price for any query. The third baseline *PDP* utilizes the similar idea of PerPricer, i.e., releases the query price over synthetic data to mislead attackers. Differently, *PDP* computes

TABLE IV
PERFORMANCE COMPARISON ON INFERRING DATA EXISTENCE

Method		*Walmart*			*Employment*			*TPC-H*			*SSB*		
		Accuracy	F1	Time (ms)	Accuracy	F1	Time (ms)	Accuracy	F1	Time (ms)	Accuracy	F1	Time (ms)
Naive	PBP	1.000	1.000	0.040	1.000	1.000	0.040	1.000	1.000	0.041	1.000	1.000	0.041
	IBP	0.999	0.999	0.039	0.978	0.989	0.039	1.000	1.000	0.039	1.000	1.000	0.039
MI	PBP	—	—	—	—	—	—	1.000	1.000	0.041	1.000	1.000	0.040
	IBP	—	—	—	—	—	—	1.000	1.000	0.038	1.000	1.000	0.039

TABLE V
PERFORMANCE COMPARISON OF MITIGATING CARDINALITY LEAKAGE

Method		*Walmart*			*Employment*			*TPC-H*			*SSB*		
		MAE	MSE	Time (s)	MAE	MSE	Time (s)	MAE	MSE	Time (s)	MAE	MSE	Time (s)
CIL	PBP	0	0	0.015	0	0	0.007	0	0	0.006	0	0	0.018
	IBP	3.815	81.49	0.086	1.805	9.585	0.046	1.3822	46.616	0.016	2.138	40.237	0.543
CIH	IBP	0.102	0.204	0.025	0.376	0.753	0.015	0.054	0.107	0.012	0.083	0.167	0.098

TABLE VI
PERFORMANCE COMPARISON OF MITIGATING CARDINALITY LEAKAGE

Method	*Walmart*			*Employment*			*TPC-H*			*SSB*		
	Δ_{MAE}	Δ_{price}	Time (s)	Δ_{MAE}	Δ_{price}	Time (s)	Δ_{MAE}	Δ_{price}	Time (s)	Δ_{MAE}	Δ_{price}	Time (s)
FP	0	10	0.040	0	10	0.040	0	10	0.041	0	10	0.043
DP	∞	4533.10	0.039	∞	2098.12	0.041	∞	1382987.34	0.039	∞	5453519.24	0.039
PDP	6.17	43.21	0.363	12.58	23.36	0.420	19.12	56.93	5.02	90.25	145.82	6.92
PerPricer	10.21	6.78	0.124	10.57	9.56	0.156	11.23	8.43	1.21	9.34	4.79	1.54

TABLE VII
PERFORMANCE COMPARISON OF MITIGATING EXISTENCE LEAKAGE

Method	*Walmart*			*Employment*			*TPC-H*			*SSB*		
	Δ_{acc}	Δ_{price}	Time (s)	Δ_{acc}	Δ_{price}	Time (Ms)	Δ_{acc}	Δ_{price}	Time (s)	Δ_{acc}	Δ_{price}	Time (s)
FP	0.325	10.00	0.040	0.268	10.00	0.043	0.239	10.00	0.041	0.367	10.00	0.045
DP	0.325	4236.12	0.045	0.268	2492.78	0.039	0.411	1499989.12	0.039	0.504	6001151.34	0.039
PDP	0.109	12.46	5.36	0.125	16.74	6.34	0.312	623.47	11.03	0.398	456.21	13.67
PerPricer	0.325	11.00	3.49	0.268	11.00	4.12	0.411	10	8.09	0.504	11.00	10.12

TABLE VIII
PERFORMANCE OF INFERRING OTHER DATA INFORMATION

Data information	*Walmart*	*Employment*	*TPC-H*	*SSB*
Maximum	0	1.81e+9	305.41	0
Minimum	0	0	0	0.45
Attribute value	0	1	0	0.003
Column distribution	1.21	0.63	0.79	1.73

the prices of a query over multiple databases (including the real one) and releases the maximal price.

The hash function in PerPricer maps cardinality k to the nearest value divisible by h, i.e., $H(k) = 1 + \lceil k/h \rceil \cdot h - k$ for $k > 0$ and $H(0) = 1 + h$, so $H(k) \leq h + 1$ for all k. We fix $h = 10$ across four datasets. Thus, overpayment is controlled by the local sensitivity ($\leq h + 1$) rather than an unbounded margin. We use five databases for *PDP*, add a fixed 10 in *FP*, and repeat each experiment ten times.

A. Evaluation on Data Inference Algorithms

In this set of experiments, we verify the effectiveness of the proposed attack algorithms in inferring data information.

Table IV reports the existence-inference results for *Naive* and MI. MI reasons over join-price correlations, so it is undefined for single-relation datasets such as *Walmart* and *Employment*, whereas both algorithms enjoy constant running time elsewhere. *Naive* achieves perfect accuracy andF1 under PBP because a zero price is a necessary and sufficient witness of an empty query answer. Under IBP, nonexistent values may still be quoted with a positive price, introducing small errors, yet large benchmarks still yield perfect accuracy thanks to the abundance of free queries that *Naive* can purchase to verify existence. MI exploits the arbitrage-free pricing constraints to recover the exact existence outcome on *TPC-H* and *SSB*. The consistently high accuracy reflects the substantial exposure risk inherent in current query-pricing markets.

The experimental results of cardinality inference (i.e., CIL and CIH algorithms) over four datasets are depicted in Table **??**. Specifically, CIL is applicable on the pricing frameworks PBP and IBP, while CIH is feasible on IBP. As observed, CIL has the zero MAE and MSE on PBP, where the price of `limit` K query becomes the highest when K is no less than the real cardinality. In contrast, since IBP prices the `limit` K query based on the randomly generated possible databases, the query price can be the highest when K is far from the real cardinality, leading to the error of cardinality inference. Furthermore, it is easy to find that CIH can derive the cardinality close to the real one over the pricing framework IBP. It attributes to that, CIH leverages the property of IBP on pricing `having` queries, whose price is unrelated to the tuple order. In summary, the performance of CIL and CIH is consistent with our theoretical findings in Theorem 2 and 3.

We assess the power of proposed attack algorithms on inferring more types of data information, including the extreme value, the value of an attribute of a specific tuple, and the column distribution. In particular, the max/min and attribute values are derived based on the existence inference algorithms, while the column distribution is obtained via cardinality inference. We calculate the mean absolute error (MAE) between the inferred and actual values, and measure the earth mover's

distance (EMD) between the inferred and actual distributions. The corresponding results under information-based pricing are reported in Table VIII. As the proposed algorithms can easily derive accurate data information from provenance-based prices (PBP), the results are omitted due to space constraints. One can find that, the inferred maximum/minimum/attribute value under IBP can be different from the real one. This is because, IBP prices each query based on query results over the real and possible databases. Hence, the inferred maximum/minimum under IBP is the max/min value among all databases, while the attribute value of a tuple is the average of those values among all databases. Besides, there is a slight difference between the inferred column distribution and the real one due to the tiny inference error of CIH. The derived data information is close to the real one in most cases, confirming the effectiveness of the proposed information inference algorithms.

B. Evaluation on PerPricer

In the set of experiments, we evaluate whether the proposed methods can mitigate the risk of information leakage.

We evaluate the performance of the mitigation methods in preventing the cardinality leakage. The experimental results are reported in Table VI where Δ_{MAE} is the increased inference error after employing the leakage mitigation method. Therefore, a larger Δ_{MAE} indicates a stronger mitigation effect, as it means the attacker's prediction of the true cardinality becomes less accurate. It is found that, *FP* cannot prevent the information leakage on data cardinality. Meanwhile, the attacker cannot infer the cardinality given the database price under *DP*, leading to infinite Δ_{MAE}. Under PerPricer and *PDP*, the attacker can infer the *wrong* information with the higher MAE. PerPricer has the limited overpayment and is more efficient than *PDP*. It confirms the usefulness of PerPricer in alleviating information leakage with acceptable overpayment and high efficiency. Our method consistently reduces cardinality leakage across all four datasets, whereas *FP* offers no improvement, i,e., $\Delta_{MAE} = 0$.

Table VII compares four defences against existence leakage. We report $\Delta_{acc} = Acc_o - Acc_m$ (larger is better because the attacker loses more accuracy) and the buyer overpayment Δ_{price}. Δ_{price} corresponds to the extra payment of data buyers under each method. A smaller Δ_{price} is desirable, as it means the mitigation achieves information protection with less overpayment, thereby maintaining higher pricing efficiency and user affordability. The *DP* and PerPricer are superior to *FP* and *PDP*, which significantly reduce the inference accuracy. *FP* still exposes prices that MI exploits, and *PDP* cannot guarantee a non-zero query price for missing values. *PDP* is also slow because it prices each query on multiple databases, while *DP* incurs the largest Δ_{price} by charging the full database. Overall, PerPricer protects existence information with bounded overpayment close to its theoretical limit.

In summary, the experiments validate the risks of data existence and cardinality leakage across real-world datasets and benchmark environments, demonstrating that adversaries can reliably infer sensitive information without financial cost. The proposed PerPricer framework efficiently mitigates these vulnerabilities through price obfuscation mechanisms.

IX. Related Work

Query pricing. Arbitrage-free pricing for SQL queries has been widely studied, including view-based schemes that charge via determining views [29], [30], provenance-based pricing that sums contributing tuples for SPJ [37], and QIRANA-style approaches that extend to SPJA via possible worlds and inspire revenue maximization [10], [15], [17], [18]. Surveys synthesize techniques and systems for data pricing [4], [38], [44], [56]. Applications span crowd sensing [2], [3], [50], [58], personal data [7], [32], [41], and IoT data [8], [24], [33], as well as graphs, models, and prediction services [11], [12], [25], [36], [45], [55]. However, none of them formally study information leakage from prices.

Side channel attacks. Prior work studies leakage from the physical execution of query processing. These works exploit timing variations [6], [13], [22], exception reporting [39], and cache behavior [47] to infer private keys, data existence, and cardinality, while Boenisch et al. [6] further demonstrate multiple channels within one system. Mitigations include data-oblivious algorithms [13], hardening database engines in error handling and optimization [43], [49], and limiting SQL functionality. These methods address execution-level channels and do not mitigate leakage from query prices, which must preserve economic meaning and cannot be assigned arbitrarily.

Overall, we make the first investigation on information leakage in query pricing systems and develop practical algorithms to infer sensitive data information at no financial cost. Additionally, we present the PerPricer framework that mitigates information leakage while preserving theoretical guarantees, maintaining both consumer cost control and pricing mechanism efficiency.

X. Conclusion

In this paper, we explore the problem of information leakage in query pricing, i.e., how the query prices tell about the data and how to prevent such leakage. We first identify two types of information leakage and propose four algorithms to infer the data existence and cardinality with high accuracy. We analyze the limitations of typical strategies for leakage mitigation and formulate the problem of leakage mitigation based on masked pricing. We further propose a framework PerPricer that perturbs the original data and releases the query price over the perturbed data. PerPricer not only addresses the information leakage with theoretical guarantee but also has the high efficiency and bounded overpayment. Extensive experiments on real-world and large-scale datasets confirm the existence of information leakage in the real world and the effectiveness of PerPricer. In the future, we intend to explore the effect of PerPricer on the customers' willingness-to-pay and revenue maximization.

XI. AI-Generated Content Acknowledgement

After completing the full technical draft, the authors used ChatGPT 5 (OpenAI) to polish prose in the narrative sections. The assistant additionally checked consistency of domain-specific terminology, acronym capitalization, math notation, and special font usage throughout the manuscript. No new scientific content, figures, tables, or experimental configurations were generated by the tool, nor was any proprietary or unpublished data uploaded. All AI-suggested edits were reviewed, verified, and selectively accepted or revised by the authors, who retain full responsibility for the final manuscript.

References

[1] Amazon. Aws marketplace, 2023.

[2] Baoyi An, Mingjun Xiao, An Liu, Xike Xie, and Xiaofang Zhou. Crowdsensing data trading based on combinatorial multi-armed bandit and Stackelberg game. In *ICDE*, pages 253–264, 2021.

[3] Baoyi An, Mingjun Xiao, An Liu, Yun Xu, Xiangliang Zhang, and Qing Li. Secure crowdsensed data trading based on blockchain. *IEEE Trans. Mob. Comput.*, 22(3):1763–1778, 2023.

[4] Santiago Andrés Azcoitia, Costas Iordanou, and Nikolaos Laoutaris. Understanding the price of data in commercial data marketplaces. In *ICDE*, pages 3718–3728, 2023.

[5] Magdalena Balazinska, Bill Howe, and Dan Suciu. Data markets in the cloud: An opportunity for the database community. *Proc. VLDB Endow.*, 4(12):1482–1485, 2011.

[6] Franziska Boenisch, Reinhard Munz, Marcel Tiepelt, Simon Hanisch, Christiane Kuhn, and Paul Francis. Side-channel attacks on query-based data anonymization. In *CCS*, pages 1254–1265, 2021.

[7] Hui Cai, Fan Ye, Yuanyuan Yang, Yanmin Zhu, Jie Li, and Fu Xiao. Online pricing and trading of private data in correlated queries. *IEEE Trans. Parallel Distributed Syst.*, 33(3):569–585, 2022.

[8] Zhipeng Cai, Xu Zheng, Jinbao Wang, and Zaobo He. Private data trading towards range counting queries in internet of things. *IEEE Trans. Mob. Comput.*, 22(8):4881–4897, 2023.

[9] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom B. Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *USENIX*, pages 2633–2650, 2021.

[10] Shuchi Chawla, Shaleen Deep, Paraschos Koutris, and Yifeng Teng. Revenue maximization for query pricing. *Proc. VLDB Endow.*, 13(1):1–14, 2019.

[11] Chen Chen, Ye Yuan, Zhenyu Wen, Guoren Wang, and Anteng Li. GQP: A framework for scalable and effective graph query-based pricing. In *ICDE*, pages 1573–1585, 2022.

[12] Lingjiao Chen, Paraschos Koutris, and Arun Kumar. Towards model-based pricing for machine learning in a data marketplace. In *SIGMOD*, pages 1535–1552, 2019.

[13] Chen Dar, Moshik Hershcovitch, and Adam Morrison. Rls side channels: Investigating leakage of row-level security protected data through query execution time. *SIGMOD*, 1(1):1–25, 2023.

[14] Smart Data. Us employment numbers by industry, 2024.

[15] Shaleen Deep and Paraschos Koutris. The design of arbitrage-free data pricing schemes. *arXiv preprint arXiv:1606.09376*, 2016.

[16] Shaleen Deep and Paraschos Koutris. The design of arbitrage-free data pricing schemes. In *ICDT*, pages 12:1–12:18, 2017.

[17] Shaleen Deep and Paraschos Koutris. QIRANA: A framework for scalable query pricing. In *SIGMOD*, pages 699–713, 2017.

[18] Shaleen Deep, Paris Koutris, and Yash Bidasaria. QIRANA demonstration: Real time scalable query pricing. *Proc. VLDB Endow.*, 10(12):1949–1952, 2017.

[19] Cynthia Dwork. Differential privacy. In *ICALP*, pages 1–12, 2006.

[20] Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. *Found. Trends Theor. Comput. Sci.*, 9(3-4):211–407, 2014.

[21] Foursquare. Fsq places data: Walmart in us, 2024, Notes: The original link was lost due to the product being taken off the shelves. Dataset is available in https://anonymous.4open.science/r/PerPricer-B638/README.md.

[22] Ariel Futoransky, Damián Saura, and Ariel Waissbein. The ND2DB attack: Database content extraction using timing attacks on the indexing algorithms. In *WOOT*, 2007.

[23] Maryam Ghasemaghaei and Goran Calic. Assessing the impact of big data on firm innovation performance: Big data is not always better data. *J. Bus. Res.*, 108:147–162, 2020.

[24] Zaobo He and Zhipeng Cai. Trading aggregate statistics over private internet of things data. *IEEE Trans. on Computers*, 73(2):394–407, 2024.

[25] Huiwen Hou, Lianpeng Qiao, Ye Yuan, Chen Chen, and Guoren Wang. A scalable query pricing framework for incomplete graph data. In *DASFAA*, pages 97–113, 2023.

[26] Hongsheng Hu, Zoran Salcic, Lichao Sun, Gillian Dobbie, Philip S. Yu, and Xuyun Zhang. Membership inference attacks on machine learning: A survey. *ACM Comput. Surv.*, 54(11s):235:1–235:37, 2022.

[27] Tao Jiang, Xu Yuan, Qiong Cheng, Yulong Shen, Liangmin Wang, and Jianfeng Ma. Fairecom: Towards proof of e-commerce fairness against price discrimination. *IEEE Trans. Dependable Secur. Comput.*, 21(4):3528–3544, 2024.

[28] Charles I Jones and Christopher Tonetti. Nonrivalry and the economics of data. *Am. Econ. Rev.*, 110(9):2819–2858, 2020.

[29] Paraschos Koutris, Prasang Upadhyaya, Magdalena Balazinska, Bill Howe, and Dan Suciu. Query-based data pricing. In *PODS*, pages 167–178, 2012.

[30] Paraschos Koutris, Prasang Upadhyaya, Magdalena Balazinska, Bill Howe, and Dan Suciu. Toward practical query pricing with query market. In *SIGMOD*, pages 613–624, 2013.

[31] Paraschos Koutris, Prasang Upadhyaya, Magdalena Balazinska, Bill Howe, and Dan Suciu. Query-based data pricing. *J. ACM*, 62(5):43, 2015.

[32] Chao Li, Daniel Yang Li, Gerome Miklau, and Dan Suciu. A theory of pricing private data. *ACM Trans. Database Syst.*, 39(4):34:1–34:28, 2014.

[33] Qinya Li, Zun Li, Zhenzhe Zheng, Fan Wu, Shaojie Tang, Zhao Zhang, and Guihai Chen. Capitalize your data: Optimal selling mechanisms for IoT data exchange. *IEEE Trans. Mob. Comput.*, 22(4):1988–2000, 2023.

[34] Bing-Rong Lin and Daniel Kifer. On arbitrage-free pricing for general data queries. *Proc. VLDB Endow.*, 7(9):757–768, 2014.

[35] Bing-Rong Lin and Daniel Kifer. Information measures in statistical privacy and data processing applications. *ACM Trans. Knowl. Discov. Data*, 9(4):28:1–28:29, 2015.

[36] Jinfei Liu, Jian Lou, Junxu Liu, Li Xiong, Jian Pei, and Jimeng Sun. Dealer: An end-to-end model marketplace with differential privacy. *Proc. VLDB Endow.*, 14(6), 2021.

[37] Xiaoye Miao, Yunjun Gao, Lu Chen, Huanhuan Peng, Jianwei Yin, and Qing Li. Towards query pricing on incomplete data. *IEEE Trans. Knowl. Data Eng.*, 34(8):4024–4036, 2022.

[38] Xiaoye Miao, Huanhuan Peng, Xinyu Huang, Lu Chen, Yunjun Gao, and Jianwei Yin. Modern data pricing models: Taxonomy and comprehensive survey. *arXiv preprint arXiv:2306.04945*, 2023.

[39] Microsoft. Security note: Side-channel attacks, 2024.

[40] Alan Nash, Luc Segoufin, and Victor Vianu. Views and queries: Determinacy and rewriting. *ACM Trans. Database Syst.*, 35(3):1–41, 2010.

[41] Chaoyue Niu, Zhenzhe Zheng, Fan Wu, Shaojie Tang, Xiaofeng Gao, and Guihai Chen. Unlocking the value of privacy: Trading aggregate statistics over private correlated data. In *SIGKDD*, pages 2031–2040, 2018.

[42] Patrick E. O'Neil, Elizabeth J. O'Neil, Xuedong Chen, and Stephen Revilak. The star schema benchmark and augmented fact table indexing. In *TPCTC*, volume 5895, pages 237–252, 2009.

[43] Andrei Paduroiu, Sungheun Wi, Yan Yan, Roni Burd, Ruhollah A Farchtchi, and Giovanni Matteo Fumarola. Membrane - safe and performant data access controls in apache spark in the presence of imperative code. *Proc. VLDB Endow.*, 17(12):3813–3826, 2024.

[44] Jian Pei. A survey on data pricing: From economics to data science. *IEEE Trans. Knowl. Data Eng.*, 34(10):4586–4608, 2020.

[45] Huanhuan Peng, Xiaoye Miao, Lu Chen, Yunjun Gao, and Jianwei Yin. Pricing prediction services for profit maximization with incomplete information. In *ICDE*, pages 1353–1365, 2023.

[46] Raluca Ada Popa, Catherine MS Redfield, Nickolai Zeldovich, and Hari Balakrishnan. Cryptdb: Protecting confidentiality with encrypted query processing. In *SOSP*, pages 85–100, 2011.

[47] Aria Shahverdi, Mahammad Shirinov, and Dana Dachman-Soled. Database reconstruction from noisy volumes: A cache side-channel attack on sqlite. In *USENIX*, pages 1019–1035, 2021.
[48] Congzheng Song and Ananth Raghunathan. Information leakage in embedding models. In *CCS*, pages 377–390, 2020.
[49] SQLShack. Methods to avoid the sql divide by zero error, 2019.
[50] He Sun, Mingjun Xiao, Yin Xu, Guoju Gao, and Shu Zhang. Privacy-preserving stable crowdsensing data trading for unknown market. In *INFOCOM*, pages 1–10, 2023.
[51] Ruiming Tang, Huayu Wu, Zhifeng Bao, Stéphane Bressan, and Patrick Valduriez. The price is right: Models and algorithms for pricing data. In *DEXA*, pages 380–394, 2013.
[52] TPC. TPC-H dataset, 2024.
[53] Grand view research. Big data market size, share & trends analysis report, 2024.
[54] Dhinakaran Vinayagamurthy, Alexey Gribov, and Sergey Gorbunov. Stealthdb: A scalable encrypted database with full SQL query support. *Proc. Priv. Enhancing Technol.*, 2019(3):370–388, 2019.
[55] Anran Xu, Zhenzhe Zheng, Qinya Li, Fan Wu, and Guihai Chen. VAP: Online data valuation and pricing for machine learning models in mobile health. *IEEE Trans. Mob. Comput.*, 23(5):5966–5983, 2024.
[56] Mengxiao Zhang, Fernando Beltrán, and Jiamou Liu. A survey of data pricing for data marketplaces. *IEEE Trans. Big Data*, 9(4):1038–1056, 2023.
[57] Xiaoping Zhang and Xusen Cheng. I lose vs. I earn: Consumer perceived price fairness toward algorithmic (vs. human) price discrimination. In *CHI*, pages 473:1–473:17, 2024.
[58] Zhenzhe Zheng, Yanqing Peng, Fan Wu, Shaojie Tang, and Guihai Chen. ARETE: On designing joint online pricing and reward sharing mechanisms for mobile data markets. *IEEE Trans. Mob. Comput.*, 19(4):769–787, 2019.